

Remove Deprecated Locale-Category Facets for Unicode from C++26

Document #: P2873R3
Date: 2025-03-16
Project: Programming Language C++
Audience: LWG
Reply-to: Alisdair Meredith
<ameredith1@bloomberg.net>
Tom Honermann
<tom@honermann.net>

Contents

1	Abstract	2
2	Revision History	2
	R3: March 2025 (post-Hagenberg mailing)	2
	R2: July 2024 (post-St. Louis mailing)	2
	R1: April 2024 (post-Tokyo mailing)	2
	R0: May 2023 (pre-Varna mailing)	2
3	Introduction	3
4	Problem Description	3
5	History	4
6	C++23 Feedback	5
	6.1 Initial LEWGI review: Telecon 2020/07/13	5
	6.2 SG16 review: Telecon 2020/07/22	5
	6.3 LEWGI consensus	5
7	Proposal	6
	7.1 Deployment experience	6
8	C++26 Review	7
	8.1 SG16 initial review: Telecon 2023/05/24	7
	8.2 SG16 second review: Telecon 2023/10/25	7
	8.3 LWG review of [LWG3767]: Kona meeting, November 2023	7
	8.4 LEWG initial review: Telecon 2026/03/05	7
	8.5 SG16 confirmation review: Telecon 2026/03/12	8
9	Wording	9
	9.1 No changes to zombie names	9
	9.2 Update library wording	9
10	Acknowledgements	10
11	References	10

1 Abstract

Several locale facets that were intended to support Unicode transcoding were added to C++11. Due to their use of `char` for UTF-8 encoded data, the facets that convert to and from UTF-8 were deprecated in C++20 when support for `char8_t` was added with the adoption of [P0482R6]. This paper proposes removing those facets from the C++ Standard Library.

2 Revision History

R3: March 2025 (post-Hagenberg mailing)

- Forwarded to LWG followed LEWG and SG16 telecons
- Recorded paper reviews
 - LEWG telecon, 2025/03/05
 - SG16 telecon, 2025/03/12
- Revised wording
 - Rebased wording onto the latest working draft, N5008

R2: July 2024 (post–St. Louis mailing)

- Added basic background on the code-conversion facets for nonexperts on iostreams
- Replaced the term “locale dependent” with the Standard term “locale-specific”
- Confirmed that gcc still does not emit a deprecation warning, even with `-Wdeprecated`
- Revised wording
 - Rebased wording onto the latest working draft, N5008
 - Wording incorporates [LWG3767], adopted at Tokyo
 - D.19 [depr.locale.category] remains since it is no longer emptied

R1: April 2024 (post-Tokyo mailing)

- Added Tom Honermann as coauthor
- Advanced to the next group for review: SG16 → LEWG
- Removed redundant subsection numbering from this section
- Revised the abstract and the “History” section
 - Noted that deprecation was motivated by the introduction of `char8_t`
 - Removed claims that deprecation was recommended by SG16; SG16 was formed after [P0482R6] was approved by EWG and LEWG
- Modified the “History” section to include additional details regarding the motivation for deprecation and the (incorrect) addition of `char8_t`-based replacements
- Added the “Deployment Experience” section with example code that would be expected to trigger a deprecation warning and updated the current implementation status accordingly
- Tested when (or if) popular Standard Library implementations warn of deprecation
- Recorded initial review feedback, recommending removal, from SG16
 - Added links to SG16 meeting summaries
- Revised wording
 - Validated wording against the latest working draft, N4971
 - Added Annex C wording
 - Updated stable label cross-reference to C++23
- Applied editorial feedback

R0: May 2023 (pre-Varna mailing)

- Initial draft of this paper

3 Introduction

At the start of the C++23 cycle, [P2139R2] tried to review each deprecated feature of C++ to see which would benefit us if actively removed and which might now be better undeprecated. Consolidating all this analysis into one place was intended to ease the (L)EWG review process but in return gave the author so much feedback that the next revision of the paper was not completed.

For the C++26 cycle, a much shorter paper, [P2863], will track the overall analysis, but for features that the author wants to actively progress, a distinct paper will decouple progress from the larger paper so that the delays on a single feature do not hold up progress on all.

This paper takes up the deprecated locale-category facets for Unicode, D.19 [depr.locale.category].

4 Problem Description

This section provides sufficient background such that nonexperts in the iostreams facility can understand the questions we must address.

In the iostreams library, facets are a customization point that enable customization of textual output to correspond to locale-specific conventions. They are used only where the framework explicitly specifies that they are called or where user-supplied extensions to the framework explicitly call them.

The code-conversion facets, specialization of `codecvt`, are used for transcoding from one text encoding to another, and the Standard uses only two of these, neither of which is deprecated. However, in [P0482R6], the Standard optimistically specified two more specializations to handle transcoding to and from Unicode sequences stored in the new (at that time) character types that would designate such an encoding through the type system.

These new `codecvt` specializations immediately run into two design issues. First, the facets system, although implemented through templates to support the iostreams framework, is a runtime customization that respects the locale installed at runtime. Conversely, the idea expressed by the Unicode character types is that of a compile-time encoding that does not vary at run time. Secondly, the standard iostreams components support only character types `char` and `wchar_t`; no support is offered for the Unicode character types, so the Standard will never specify a use of these facets. Extending Unicode support into iostreams would take considerable effort, and currently the Committee has shown no appetite for the extension method of adopting the Unicode character types and their compile-time encoding throughout the current iostreams facility.

The presence of these (poorly specified) facets, however, obstructs SG16 from actively working to better support I/O with the Unicode character types. SG16 strongly prefers to remove these facets, as soon as possible, to clear the way for a clean new design.

In practice, the Standard uses only two instantiations of the `codecvt` facet, and all others are meaningless outside of users extending the framework themselves. The instantiation `std::codecvt<char, char, mbstate_t>` exists and can be called but has no effect since it passes through all text unaltered. The instantiation `std::codecvt<wchar_t, char, mbstate_t>` supports transcoding from wide characters to narrow characters. The Standard itself neither instantiates nor uses `std::codecvt<wchar_t, char, mbstate_t>!`

5 History

The deprecated locale facets that convert to and from UTF-8 were added as part of the initial basic support for Unicode types in C++11 by [N2238]. However, they were introduced with inconsistent behavior relative to the corresponding `wchar_t` facet with respect to the encoding used for the `char`-typed side of the conversion. The `wchar_t` specialization converts to and from the locale-specific multibyte encoding while the (now-deprecated) `char16_t` and `char32_t` facets convert to and from UTF-8. This behavior was an odd choice, not just for consistency but because the specified behavior is unaffected by the choice of locale. Further, their addition was arguably unnecessary since the C++ Standard does not require support for streams of type `char16_t` or `char32_t`. (A previous revision of [N2238], [N2035] had proposed adding support for such streams, but that proposal was abandoned, based on LEWG feedback, after the first revision.)

At any rate, the result was two interfaces that associated different character encodings with type `char`. Prevention of this kind of encoding confusion was part of the motivation for the addition of `char8_t` in [P0482R6]. That paper deprecated these facets in favor of new ones that use `char8_t` for UTF-8.

Unfortunately, the [P0482R6] author failed to appreciate how the `std::codecvt` facets are actually intended to be used. Within the C++ Standard, they are used only by `std::basic_filebuf` (31.10.3 [filebuf]) and the `std::filesystem::path` constructor that accepts an argument of type `std::locale` (31.12.6.5.1 [fs.path.construct]) and are used only to convert streams or sequences of `wchar_t` to the sequence of `char` elements that are the contents or names of files. The new `char8_t`-based facets don't actually function as replacements because the existing use was fundamentally limited to use of (`char`-based) file streams and names. Since the Standard doesn't specify file streams of type `char8_t`, the newly added `char8_t`-based facets serve no actual purpose. (`std::filesystem::path` uses an unspecified conversion method for `char8_t`, `char16_t`, and `char32_t`; 31.12.6.3.2 [fs.path.type.cvt].) [LWG3767] now tracks their deprecation and future removal, and the resolution for this issue was adopted at the Tokyo 2024 meeting.

Thus, the status quo is as follows.

- The original addition of the deprecated facets was poorly motivated.
- The specified behavior of the deprecated facets is inconsistent and contrary to the purpose of such facets to provide locale-specific behavior.
- The facets have been deprecated since C++20.
- If the Standard were to add support for streams of type `char16_t` or `char32_t`, the existing behavior does not match the desired behavior (to convert to a locale-specific encoding).

6 C++23 Feedback

6.1 Initial LEWGI review: Telecon 2020/07/13

Discussion was broadly in favor of removal from the C++23 specification and reliance on library vendors to maintain source compatibility as long as needed. However, LEWGI explicitly requested that the author confer with SG16 to determine if that study group is aware of any reason to further delay removing this deprecated facility.

6.2 SG16 review: Telecon 2020/07/22

See <https://github.com/sg16-unicode/sg16-meetings/blob/master/README-2020.md#july-22nd-2020>.

SG16 is concerned that `std::codecvt` in general has poor error-handling facilities, especially when dealing with encodings that may take multiple code units to express a code point and thus have more cause to report on malformed inputs. The specific facets in D.19 [[depr.locale.category](#)] are an obstacle to putting a minimally useful replacement into the Standard because they reserve the good names but have poor semantics. The usual safety net of the zombie-names clause does not apply since we will want code to fail to compile for at least one release to thus introduce a replacement with the same names but more appropriate semantics. Concern was raised about removing a feature deprecated only as recently as the current C++20 Standard.

Polling showed no consensus to recommend the removal for C++23 but no objection to that removal either.

6.3 LEWGI consensus

SG16 has no objection; remove this feature from C++23.

7 Proposal

Remove deprecated locale-category facets for Unicode from C++26.

7.1 Deployment experience

The following example suffices for a deprecation warning to be emitted from popular current implementations when compiling in C++20 mode. See <https://godbolt.org/z/5Yodo9KeW>.

```
#include <locale>
struct user_codecvt_c16 : public std::codecvt<char16_t, char, std::mbstate_t> {};
struct user_codecvt_c32 : public std::codecvt<char32_t, char, std::mbstate_t> {};
struct user_codecvt_byname_c16 : public std::codecvt_byname<char16_t, char, std::mbstate_t> {};
struct user_codecvt_byname_c32 : public std::codecvt_byname<char32_t, char, std::mbstate_t> {};
```

- libc++: First warns in Clang 12 (released on 2021-04-14)
- libstdc++: Does not warn in the latest release (last tested against gcc 14.1)
- MSVC: First warns with /W3 or higher in MSVC v19.22 (VS 2019 version 16.2.3 released on 2019-08-20); does not warn by default or with /W2 or lower in the latest release

8 C++26 Review

8.1 SG16 initial review: Telecon 2023/05/24

See <https://github.com/sg16-unicode/sg16-meetings/blob/master/README-2023.md#may-24th-2023>.

Tom Honermann explained the history and motivation behind the deprecation. SG16 then endorsed removing these facets from C++26.

8.2 SG16 second review: Telecon 2023/10/25

See <https://github.com/sg16-unicode/sg16-meetings/blob/master/README-2023.md#october-25th-2023>.

The proposal was reviewed again in light of [LWG3767], which deprecates the equivalent facets that convert to `char8_t` and were thought to be the intended replacement for the deprecated facets. We considered *undeprecating* these original facets since their replacement had not worked out.

However, further discussion demonstrated that the whole notion of using these facets, which are supposed to be locale-specific, to convert from one Unicode encoding to another, using strict typing that ignores locale, was flawed. The observation was made that we are missing the facet to convert from `char8_t` to `char` if we want to continue maintaining these facets, and we do not want to be delving deeper.

A clear preference was indicated to remove these facets that are occupying the space in the library where a well-specified replacement should go; we do not feel comfortable providing a replacement in that space until at least one Standard cycle has passed without these facets, minimizing the risk that code would silently change meaning when users update to a future Standard.

Note that all discussion is predicated on a future library proposal to support Unicode types in iostreams, where they are explicitly *not* supported today. Absent such a proposal, these facets serve no purpose and are actively harmful due to their poor semantics.

The recommendation remains to pursue removal as proposed herein.

8.3 LWG review of [LWG3767]: Kona meeting, November 2023

This resolution for this issue was adopted at the Tokyo 2024 meeting, meaning this paper is no longer proposing the removal of the whole of D.19 [[depr.locale.category](#)] because more facets have been deprecated for the C++26 Standard.

8.4 LEWG initial review: Telecon 2026/03/05

After the deprecated feature was described, concerns were raised that this paper removes a feature without providing a replacement. It was pointed out that the current feature is problematic by trying to impose a compile-time encoding through a feature whose whole purpose is to support encoding through the locale installed at runtime — SG16 is moving away from using the Unicode character types in this way and is working on a better way to handle Unicode text that would be not be based on iostreams. These facets are an attractive nuisance that do nothing but lead any potential users astray.

Standard library can continue to offer these facets as vendor extensions if they so desire, allowing our library implementers to apply a policy of phased removal at a rate that is comfortable for their users.

Two polls were taken.

POLL: Forward “P2873R2: Remove Deprecated locale category facets for Unicode from C++26” to LWG for C++29 (pending SG16 decision).

SF	F	N	A	SA
11	7	0	0	0

Outcome: Unanimous consensus in favor

POLL: Forward “P2873R2: Remove Deprecated locale category facets for Unicode from C++26” to LWG with a recommendation to apply for C++26 (if possible) (pending SG16 decision).

SF	F	N	A	SA
9	7	0	0	0

Outcome: Strong consensus in favor

8.5 SG16 confirmation review: Telecon 2026/03/12

Concerns were raised to verify the claim that vendors explicitly have permission to continue supplying the removed template specializations among their set of additional facets, retaining the same meaning while enabling a phased withdrawal of the facility from their customers.

[[CHAIN OF REASONING THAT THESE ADDITIONAL SPECIALIZATIONS HAVE VENDOR SPECIFIC SEMANTICS]]

Poll: Forward to LWG as approved by LEWG (take precise wording from meeting minuted)

SF	F	N	A	SA
2	4	0	0	0

9 Wording

Make the following changes to the C++ Working Draft. All wording is relative to [N5008], the latest draft at the time of writing.

9.1 No changes to zombie names

All the entities being struck are overloads of identifiers that retain their original meaning, so no new names need to be added to 16.4.5.3.2 [zombie.names].

9.2 Update library wording

C.1.8 [diff.cpp23.depr] Annex D: Compatibility features

× **Affected subclause:** D.19 [depr.locale.category]

Change: Remove `std::codecvt` locale facets that convert between `char16_t` or `char32_t` and `char`.

Rationale: The facets have been deprecated since C++ 2020 and have semantics that do not match their use as a locale-specific character conversion facility since they are specified to always convert to and from UTF-8 rather than the locale-specific multibyte encoding. Additionally, the C++ standard does not require support for streams of type `char16_t` or `char32_t`, so these locale facets need not be required by the C++ standard library.

Effect on original feature: A valid C++ 2023 program that uses any of the following template specializations may become ill-formed:

```
— codecvt<char16_t, char, mbstate_t>,
— codecvt<char32_t, char, mbstate_t>,
— codecvt_byname<char16_t, char, mbstate_t>, or
— codecvt_byname<char32_t, char, mbstate_t>
```

D.19 [depr.locale.category] Deprecated locale category facets

- ¹ The `ctype` locale category includes the following facets as if they were specified in Table 108 of 28.3.3.1.2.1 [locale.category].

```
codecvt<char16_t, char, mbstate_t>
codecvt<char32_t, char, mbstate_t>
codecvt<char16_t, char8_t, mbstate_t>
codecvt<char32_t, char8_t, mbstate_t>
```

- ² The `ctype` locale category includes the following facets as if they were specified in Table 109 of 28.3.3.1.2.1 [locale.category].

```
codecvt_byname<char16_t, char, mbstate_t>
codecvt_byname<char32_t, char, mbstate_t>
codecvt_byname<char16_t, char8_t, mbstate_t>
codecvt_byname<char32_t, char8_t, mbstate_t>
```

- ³ The following class template specializations are required in addition to those specified in 28.3.4.2.5 [locale.codecvt]. The specializations ~~`codecvt<char16_t, char, mbstate_t>`~~ and `codecvt<char16_t, char8_t, mbstate_t>` converts between the UTF-16 and UTF-8 encoding forms, and the specializations ~~`codecvt<char32_t, char, mbstate_t>`~~ and `codecvt<char32_t, char8_t, mbstate_t>` converts between the UTF-32 and UTF-8 encoding forms.

10 Acknowledgements

Thanks to Michael Park for the pandoc-based framework used to transform this document's source from Markdown.

Thanks to Lori Hughes for reviewing this paper and providing editorial feedback.

11 References

- [LWG3767] Victor Zverovich. `codecvt<charN_t, char8_t, mbstate_t>` incorrectly added to locale.
<https://wg21.link/lwg3767>
- [N2035] Matthew Austern. 2006-05-23. Minimal Unicode support for the standard library.
<https://wg21.link/n2035>
- [N2238] Matthew Austern. 2007-04-17. Minimal Unicode support for the standard library (revision 3).
<https://wg21.link/n2238>
- [N5008] Thomas Köppe. Working Draft, Programming Languages — C++.
<https://wg21.link/n5008>
- [P0482R6] Tom Honermann. 2018-11-09. `char8_t`: A type for UTF-8 characters and strings (Revision 6).
<https://wg21.link/p0482r6>
- [P2139R2] Alisdair Meredith. 2020-07-15. Reviewing Deprecated Facilities of C++20 for C++23.
<https://wg21.link/p2139r2>
- [P2863] Alisdair Meredith. Review Annex D for C++26.
<https://wg21.link/p2863>